

Chapter 5

Glue Types and Univalence

同値な型は等しいか。たとえば Theorem 62 の $\mathbb{Z}$ と Int は、互いに逆な写像で結ばれているという意味で「同じ型」であるが、この 2 つを結ぶ $\text{Path type}_i \mathbb{Z} \text{Int}$ を作る手段を、本書はまだもっていない。 $\langle i \rangle$ で型どうしの path を作るには、 i に依存する型の式が要るが、これまでの型構成子はどれも「両端で同じ構成子」の線しか作れないからである。本章で導入する *Glue* 型は、まさにこの手段を与える：同値 $e : T \simeq A$ を「境界の材料」として型の線を作る構成子であり、そこから *univalence*—同値ならば path で結ばれる—が定理として従う。

まず同値の概念を定義する。

Definition 67 (Contractibility, Fibers, and Equivalences). $A, B : \text{type}$, $f : A \rightarrow B$, $b : B$ に対して：

$$\begin{aligned} \text{isContr}(A) &\stackrel{\text{def}}{=} (x : A) \times (y : A) \rightarrow \text{Path } A \, x \, y \\ \text{fib}(f, b) &\stackrel{\text{def}}{=} (x : A) \times \text{Path } B \, (f \, x) \, b \\ \text{isEquiv}(f) &\stackrel{\text{def}}{=} (b : B) \rightarrow \text{isContr}(\text{fib}(f, b)) \\ A \simeq B &\stackrel{\text{def}}{=} (f : A \rightarrow B) \times \text{isEquiv}(f) \end{aligned}$$

$e : A \simeq B$ の関数成分を $\pi_1(e)$ と書く。

$\text{isContr}(A)$ は「 A には元があり、他のすべての元がそれと path で結ばれる」— A が 1 点に潰れる—ことをいう。 $\text{fib}(f, b)$ は b の逆像（とその witness）であり、 $\text{isEquiv}(f)$ は「すべての逆像がちょうど 1 点」、すなわち f が全単射の homotopy 版であることをいう。恒等写像が同値であること ($\text{idEquiv}(A) : A \simeq A$) は、 $\text{fib}(\text{id}, b) = (x : A) \times \text{Path } A \, x \, b$ が singleton であり、その可縮性を Section 2.2 で示してあることから従う。また、互いに逆な写像の組（往復がそれぞれ path で恒等に等しいもの）から isEquiv が構成できることが知られている（証明は The Univalent Foundations Program 2013 の §4.4 を参照。以下この事実を用いる）。これにより、Theorem 62 で示した往復の path の組は、正式に $\mathbb{Z} \simeq \text{Int}$ の元を与える。

Definition 68 (Glue Types).

$$\frac{\Gamma \vdash \varphi : \mathbb{F} \quad \Gamma \vdash A : \text{type} \quad \Gamma, \varphi \vdash T : \text{type} \quad \Gamma, \varphi \vdash e : T \simeq A}{\Gamma \vdash \text{Glue}[\varphi \mapsto (T, e)] A : \text{type}} \text{ (GlueF)}$$

$$\frac{\Gamma, \varphi \vdash t : T \quad \Gamma \vdash a : A \quad \Gamma, \varphi \vdash \pi_1(e) t = a : A}{\Gamma \vdash \text{glue}[\varphi \mapsto t] a : \text{Glue}[\varphi \mapsto (T, e)] A} \text{ (GlueI)}$$

$$\frac{\Gamma \vdash b : \text{Glue}[\varphi \mapsto (T, e)] A}{\Gamma \vdash \text{unglue } b : A} \text{ (GlueE)}$$

さらに、次の ∂ -等値性が成り立つ：

$$\begin{aligned} \Gamma, \varphi \vdash \text{Glue}[\varphi \mapsto (T, e)] A &=_{\partial} T : \text{type} \\ \Gamma, \varphi \vdash \text{glue}[\varphi \mapsto t] a &=_{\partial} t : T \quad \Gamma, \varphi \vdash \text{unglue } b =_{\partial} \pi_1(e) b : A \end{aligned}$$

また、 β 簡約と η 規則：

$$\begin{aligned} \text{unglue}(\text{glue}[\varphi \mapsto t] a) &\rightarrow_{\beta} a & (\beta) \\ \text{glue}[\varphi \mapsto b](\text{unglue } b) &= b : \text{Glue}[\varphi \mapsto (T, e)] A & (\eta) \end{aligned}$$

直観的には、 $\text{Glue}[\varphi \mapsto (T, e)] A$ は「extent φ の上では T 、それ以外では A 」を、同値 e で貼り合わせた型である。第 1 の ∂ -等値性が「 φ 上では T そのもの」であることを述べ、 unglue は貼り合わせを剥がして A 側の成分を取り出す（ φ 上では b 自身が T の元なので、第 3 の ∂ -等値性のとおり e で送った $\pi_1(e) b$ が A 側の成分である）。（ GlueI ）の第 3 前提は貼り合わせの整合性— T 側の材料 t を e で送ると A 側の材料 a に一致すること—を要求する。なお $\beta \cdot \eta$ は導入と除去の detour に関する規則であり、Section 3.6 の分類がここでも維持されていることを確認してほしい。

Glue 型の κ -簡約 (hcomp と transp) は本体系で最も込み入った部分であり、その構成には Section 3.1 で導入した量子子消去 $\forall i$ が本質的に用いられる—あの操作の出番がここである。本書では規則の存在を認めるにとどめ、詳細は Cohen et al. (2018) の §6.2 に譲る。また、universe type_i 自身の Kan 構造（型どうしの path に沿った $\text{hcomp} \cdot \text{transp}$ ）も Glue 型を用いて与えられる（同 §7.1）。以下ではこれらを既知として用いる。

Definition 69 (Univalence Map). $\Gamma \vdash e : A \simeq B$ に対して：

$$\begin{aligned} \text{ua}(e) &\stackrel{\text{def}}{=} \langle i \rangle \text{Glue}[(i = 0) \mapsto (A, e), (i = 1) \mapsto (B, \text{idEquiv}(B))] B \\ &: \text{Path type}_i A B \end{aligned}$$

端点を検算しよう。 $[0/i]$ では面公式 $(i = 0)$ が $1_{\mathbb{F}}$ となるから、第 1 の ∂ -等値性と Lemma 27 より

$$\text{Glue}[1_{\mathbb{F}} \mapsto (A, e)] B =_{\partial} A$$

すなわち $\text{ua}(e) 0 = A$ である。同様に $[1/i]$ では $(i = 1)$ が $1_{\mathbb{F}}$ となり $\text{ua}(e) 1 = B$ である。よって $\text{ua}(e)$ はたしかに A から B への型の path である（ $\text{Path type}_i A B$ の形成には、 type_i 自身が type_{i+1} の元であるという本書の sort の階層を用いている）。

さらに、 $\text{ua}(e)$ に沿った移送は e の関数成分の適用に一致する：任意の $a : A$ に対して

$$\vdash \text{Path } B (\text{transp}^i (\text{ua}(e) i) 0_{\mathbb{F}} a) (\pi_1(e) a) \text{ true}$$

が成り立つ ($\text{ua}\beta$; Glue の transp の κ -簡約から従う。判断的等値性としては成り立たない)。すなわち、「同値で作った path に沿って運ぶ」ことは「同値を適用する」ことに path の意味で等しい。

Theorem 70 (Univalence). 任意の $A, B : \text{type}_i$ に対して、 transp を用いて構成される標準的な写像 $\text{pathToEquiv} : \text{Path type}_i A B \rightarrow (A \simeq B)$ は同値である。すなわち、以下が成り立つ。

$$\vdash (\text{Path type}_i A B) \simeq (A \simeq B) \text{ true}$$

ua が pathToEquiv の逆を与えることによる。証明は Cohen et al. (2018) の §7.2 を参照。homotopy type theory では univalence は公理として仮定される (The Univalent Foundations Program 2013) のに対し、cubical type theory ではこのように**定理**であり、しかも $\text{ua}(e)$ は Glue 型の具体的な項なので、それに沿った移送が (κ -簡約により) 計算する。これが「univalence の構成的解釈」の意味である。

■**応用： $\mathbb{Z}$ 上の型値再帰** Remark 66 で先送りにした問題に答えよう。述語 $P : \mathbb{Z} \rightarrow \text{type}$ を recursion principle で定義するには、 $\text{cancel } ab$ の場合に型どうしの path $\text{Path type } (P(\text{diff } ab)) (P(\text{diff } (s(a)) (s(b))))$ を与える必要があった。 ua がこれを可能にする。例として「零である」という述語を定義する。

まず、 s が path の同値を誘導することを見る。 $a, b : \mathbb{N}$ に対して

$$\text{sucPathEquiv}(a, b) : (\text{Path } \mathbb{N} a b) \simeq (\text{Path } \mathbb{N} (s(a)) (s(b)))$$

を構成する。関数成分は $f \stackrel{\text{def}}{=} \lambda p. \langle i \rangle s(pi)$ 、逆写像は前者関数 pd (natrec で定義でき、 $\text{pd}(s(n)) = n$ は判断的に成り立つ) を用いて $g \stackrel{\text{def}}{=} \lambda q. \langle i \rangle \text{pd}(qi)$ とする (qi の端点は $s(a), s(b)$ だから、 $g(q)$ の端点は a, b である)。往復のうち $g(f(p)) = \langle i \rangle \text{pd}(s(pi)) = p$ は判断的に成り立つ。もう一方の $f(g(q))$ と q は、同じ端点をもつ $\mathbb{N}$ の path どうしであるから、 $\mathbb{N}$ が集合であること (Section 4.6 で用いた決定可能性と Hedberg の定理による) により path で結ばれる。互いに逆な写像の組であるから、Definition 67 の後に注意した事実により同値が得られる。

これを用いて：

$$\begin{aligned} \text{isZero}_{\mathbb{Z}}(\text{diff } ab) & \stackrel{\text{def}}{=} \text{Path } \mathbb{N} a b \\ \text{isZero}_{\mathbb{Z}}(\text{cancel } abi) & \stackrel{\text{def}}{=} \text{ua}(\text{sucPathEquiv}(a, b)) i \end{aligned}$$

(hcomp の場合は universe の hcomp —Glue により与えられる—による。) $\text{ua}(\text{sucPathEquiv}(a, b))$ の端点はまさに $\text{Path } \mathbb{N} a b$ と $\text{Path } \mathbb{N} (s(a)) (s(b))$ であるから、recursion principle の証明義務が満たされている。こうして $\text{isZero}_{\mathbb{Z}}(0_{\mathbb{Z}}) = \text{Path } \mathbb{N} 0 0$ は inhabited、 $\text{isZero}_{\mathbb{Z}}(\text{diff } (s(n)) 0) = \text{Path } \mathbb{N} (s(n)) 0$ は ($\mathbb{N}$ の等値性の決定可能性より) empty である。正值性 $\mathbb{Z}^+$ や順序 $<$ も、 $\mathbb{N}$ 上の対応する述語と「 s による同値」を与えれば、まったく同じ手筋で定義できる。これが Remark 66 の「直接の道」であり、Int 経由の移送の道と比べてみてほしい。

History and Further Reading

Glue 型と univalence の証明は Cohen et al. (2018) の §6–7 による（本章の提示は $\text{hcomp} / \text{transp}$ 分解に合わせて調整した）。Voevodsky による univalence 公理の定式化と simplicial set モデル、およびそれが構成的モデルの探求を促した経緯については、Chapter 1 と Chapter 3 の History を参照。

関数外延性や商型を構成的に扱う別の系譜として、observational type theory (Altenkirch et al., 2007) および setoid によるアプローチ (Hofmann, 1995) がある。cubical type theory は、これらが個別に扱ってきた問題（関数外延性・商・univalence）を単一の interval の機構で一挙に解決する体系とみることができる。cubical 手法全般の survey としては Mörtberg (2021) がある。

Bibliography

- Altenkirch, T., C. McBride, and W. Swierstra. (2007) “Observational equality, now!”, In *Proceedings of the 2007 Workshop on Programming Languages meets Program Verification (PLPV '07)*, A. Stump and H. Xi (eds.). pp.57–68.
- Altenkirch, T. and L. Scoccola. (2020) “The Integers as a Higher Inductive Type”, In *Proceedings of the 35th Annual ACM/IEEE Symposium on Logic in Computer Science*. pp.67–73, ACM.
- Angiuli, C., G. Brunerie, T. Coquand, R. Harper, K.-B. Hou (Favonia), and D. R. Licata. (2021) “Syntax and models of Cartesian cubical type theory”, *Mathematical Structures in Computer Science* **31**(4), pp.424–468.
- Angiuli, C., R. Harper, and T. Wilson. (2017) “Computational higher-dimensional type theory”, In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages*. pp.680–693, ACM.
- Awodey, S. and M. A. Warren. (2009) “Homotopy theoretic models of identity types”, *Mathematical Proceedings of the Cambridge Philosophical Society* **146**(1), pp.45–55.
- Bekki, D. (2014) “Representing Anaphora with Dependent Types”, In *Proceedings of Logical Aspects of Computational Linguistics (8th international conference, LACL2014, Toulouse, France), LNCS 8535*, N. Asher and S. V. Soloviev (eds.). pp.14–29, Springer, Heidelberg.
- Bekki, D. and K. Mineshima. (2017) “Context-Passing and Underspecification in Dependent Type Semantics”, In: S. Chatzikyriakidis and Z. Luo (eds.): *Modern Perspectives in Type-Theoretical Semantics*, Vol. 98 of *Studies in Linguistics and Philosophy*. Springer, pp.11–41.
- Bezem, M., T. Coquand, and S. Huber. (2014) “A Model of Type Theory in Cubical Sets”, In *Proceedings of 19th International Conference on Types for Proofs and Programs (TYPES 2013)*, R. Matthes and A. Schubert (eds.), Vol. 26 of *Leibniz International Proceedings in Informatics (LIPIcs)*. pp.107–128, Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
- Cavallo, E. and R. Harper. (2019) “Higher Inductive Types in Cubical Computational Type Theory”, *Proceedings of the ACM on Programming Languages* **3**(POPL), pp.1:1–1:27.
- Cohen, C., T. Coquand, S. Huber, and A. Mörtberg. (2018) “Cubical Type Theory: A Constructive Interpretation of the Univalence Axiom”, In *Proceedings of 21st International Conference on Types for Proofs and Programs (TYPES 2015)*, T. Uustalu (ed.), Vol. 69 of *Leibniz International Proceedings in Informatics (LIPIcs)*. pp.5:1–5:34, Schloss Dagstuhl – Leibniz-Zentrum für Informatik. Preprint: arXiv:1611.02108.
- Coquand, T., S. Huber, and A. Mörtberg. (2018) “On Higher Inductive Types in Cubical Type Theory”, In *Proceedings of LICS '18: Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science*. pp.255–264.

- Hofmann, M. (1995) “Extensional concepts in intensional type theory”, Ph.D. thesis, University of Edinburgh.
- Hofmann, M. and T. Streicher. (1998) “The groupoid interpretation of type theory”, In: G. Sambin and J. M. Smith (eds.): *Twenty-Five Years of Constructive Type Theory (Venice, 1995)*, Oxford Logic Guides, vol.36. New York, Oxford University Press.
- Huber, S. (2019) “Canonicity for Cubical Type Theory”, *Journal of Automated Reasoning* **63**(2), pp.173–210.
- Kapulkin, K. and P. L. Lumsdaine. (2021) “The simplicial model of Univalent Foundations (after Voevodsky)”, *Journal of the European Mathematical Society* **23**(6), pp.2071–2126.
- Lumsdaine, P. L. and M. Shulman. (2020) “Semantics of higher inductive types”, *Mathematical Proceedings of the Cambridge Philosophical Society* **169**(1), pp.159–208.
- Martin-Löf, P. (1975) “An intuitionistic theory of types”, In: H. E. Rose and J. Shepherdson (eds.): *Logic Colloquium ’73*. Amsterdam, North-Holland, pp.73–118.
- Martin-Löf, P. (1984) *Intuitionistic Type Theory*, Vol. 17. Naples, Italy: Bibliopolis. Sambin, Giovanni (ed.).
- Mörtberg, A. (2021) “Cubical methods in homotopy type theory and univalent foundations”, *Mathematical Structures in Computer Science* **31**(10), pp.1147–1184.
- Orton, I. and A. M. Pitts. (2016) “Axioms for Modelling Cubical Type Theory in a Topos”, In *Proceedings of 25th EACSL Annual Conference on Computer Science Logic (CSL 2016)*, J.-M. Talbot and L. Regnier (eds.), Vol. 62 of *Leibniz International Proceedings in Informatics (LIPIcs)*. pp.24:1–24:19, Schloss Dagstuhl – Leibniz-Zentrum für Informatik. Journal version: *Logical Methods in Computer Science* 14(4:23), 2018.
- Sterling, J. and C. Angiuli. (2021) “Normalization for Cubical Type Theory”, In *Proceedings of 36th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS 2021)*. pp.1–15, IEEE.
- Streicher, T. (1993) “Investigations into Intensional Type Theory”, Ph.D. thesis, Habilitationsschrift, Ludwig-Maximilians-Universität.
- The Univalent Foundations Program. (2013) *Homotopy Type Theory: Univalent Foundations of Mathematics*. <https://homotopytypetheory.org/book>.
- Vezzosi, A., A. Mörtberg, and A. Abel. (2019) “Cubical Agda: A Dependently Typed Programming Language with Univalence and Higher Inductive Types”, *Proceedings of the ACM on Programming Languages* **3**, pp.87:1–87:29.